

Problem 1

1-1

$T = \text{"b"}, n = 1$
 $P = \text{"aaaa"}, m = 4$
 $\pi = (0, 1, 2, 3)$

Since the algo sets $q = j$ in each iteration for $j = 0$ to $m - 1$, in the while loop, q will have to move back j times, so the total iteration ran will be $0 + 1 + 2 + 3 = 6 > 1 \times 4$. In general, this process takes $0 + 1 + 2 + \dots + m - 1 = O(m^2)$ time.

Additionally, the inner for loop can run n times per outer loop iteration to increase q , which takes $O(mn)$ time. Therefore, the time complexity of this algorithm is $O(m^2) + O(mn) + O(c) = O(m^2 + mn + c)$.

1-2.a

```
1 computePrefixAutomation(P)
2     m = |P|
3     pi = computePrefixFunction(P)
4     A = 2D array with (m + 1) × |Σ| entries
5     for q = 0 to m
6         for c in Σ
7             if q == 0 and P[q + 1] != c
8                 A[q][idx(c)] = 0
9             else if q == m or P[q + 1] != c
10                A[q][idx(c)] = A[pi[q]][idx(c)]
11            else
12                A[q][idx(c)] = q + 1
13     return A
```

Explanation:

The idea of the algorithm as the following: given that the previous q characters are matched, if the next character is c , how long will the prefix-suffix pair be at this moment. So $A[q][\text{idx}(c)] = q + 1$ if there's a match. Otherwise, we fall back onto $\pi[q]$ and how it handles character c when $q = \pi[q]$.

Time complexity:

The inner loop take $O(|\Sigma|)$ time to iterate through every character in the set, and the outer loop take will run m iterations, the overall time complexity is $O(m|\Sigma|)$.

Space complexity:

The bottleneck of space complexity is the 2D array, which sizes $(m + 1) \times |\Sigma|$, so the space complexity is $O(m|\Sigma|)$.

1-2.b

```
1 repetitiveStringMatching(T, P, c)
2     n = |T|
3     m = |P|
4     pi = Compute-Prefix-Function(P)
5     A = computePrefixAutomation(P)
6     f[0:m-1] = new array
7     g[0:m-1] = new array
8     for j = 0 to m - 1
9         q = j
```

```

10     cnt = 0
11     for i = 1 to n
12         q = A[q][idx(T[i])]
13         if q == m
14             cnt++
15             q = pi[q]
16     f[j] = q
17     g[j] = cnt
18     q = 0
19     cnt = 0
20     for i = 1 to c
21         cnt = cnt + g[q]
22         q = f[q]
23     return cnt

```

Explanation:

We can pre-compute the resulting q given the current number of matched characters and the next input character. By doing so, when deriving the values of f and g arrays, we can find the next state q in $O(1)$ time, avoiding the while loop.

Time complexity:

As elaborated in the previous question, deriving the `computePrefixAutomation` function takes $O(m|\Sigma|)$ time. Because accessing the values in A take $O(1)$ time, the total time spend in the double for loop takes $O(mn)$ time. And the final for loop takes $O(c)$ time, resulting in an overall time complexity of $O(m|\Sigma| + mn + c)$.

Space complexity:

The `computePrefixAutomation` function creates a 2D array with $(m + 1) \times |\Sigma|$ entries, so the space complexity is $O(m|\Sigma|)$. Array f and g takes $O(m)$ space, so the overall space complexity is $O(m|\Sigma|)$.

1-3

```

1 #the '^' sign stand for exponential
2 anagramMatching(T, P)
3     m = P.length
4     n = T.length
5     B = m + 1
6     MOD_NUM = pow(10, 16) + 61
7     hashValueP = hashValueT = 0
8     cnt = 0
9     for i from 1 to m
10         hashValueP = (hashValueP + B ^ getValue(P[i]) % MOD_NUM) % MOD_NUM
11         hashValueT = (hashValueT + B ^ getValue(T[i]) % MOD_NUM) % MOD_NUM
12     if hashValueP == hashValueT
13         cnt++
14     for i from m + 1 to n
15         prevCharVal = B ^ getValue(T[i - m]) % MOD_NUM
16         newCharVal = B ^ getValue(T[i]) % MOD_NUM
17         hashValueT = (hashValueT + newCharVal - prevCharVal + MOD_NUM) % MOD_NUM
18         if hashValueP == hashValueT
19             cnt++
20     return cnt

```

Explanation:

The hash function is $H(S) = \sum \text{count}(c) \times B^{\text{getValue}(c)}$, and $B = m + 1$ is a number greater than the length of P , preventing any character from carrying over into the next base position. By

doing so, the order of the characters does not matter, and different string will produce different hash values. The rest of the logic is the same as the Rabin-Karp algorithm.

Time complexity:

Since calculating x^y takes $O(\log y)$ time, calculating the exponential for one hashValue takes $O(\log |\Sigma|)$ time, for the exponent depends on the size of the set. We will calculate one hash value for every character in T and P , resulting in a $O((m + n) \log |\Sigma|)$ time complexity.

Space complexity:

Since we derive the hash value by rolling through the strings, every operation in the algorithm takes $O(1)$ space, resulting in $O(1)$ overall space complexity.

1-4

```

1 prefixSuffix(S)
2   n = |S|
3   pi = createArray(size = n)
4   pi[1] = 0
5   validLen = createArray(size = n, all elements initialized to 0)
6   freq = createArray(size = n, all elements initialized to 1)
7   k = 0
8   for q = 2 to n
9       while k > 0 and S[k + 1] != S[q]
10          k = pi[k]
11          if S[k + 1] == S[q]
12              k = k + 1
13          pi[q] = k
14   for i = n downto 1
15       if pi[i] != 0
16           freq[pi[i]] += freq[i]
17   q = n
18   i = 0
19   while q != 0
20       validLen[i++] = q
21       q = pi[q]
22   cnt = 0
23   for i = 1 to n
24       if validLen[i] == 0
25           return cnt
26       cnt = cnt + freq[validLen[i]]
27   return cnt

```

Explanation:

We create a `validLen` array to store the length of the subarray that meets the requirement, and a `freq` array with `freq[i]` storing the total appearances of $S[1 : i]$ in the array. Since every subarray will appear at least once, we initialize every element to 1. Since $S[1 : \pi[q]]$ is the longest subarray in $S[1 : q]$, we utilize this trait to find every value in the `freq` array. Then we recursively call the π values to find out all substrings that meet the requirement of being a prefix and suffix, and we sum up the total appearances of these substrings.

Time complexity:

The prefix computation process takes $O(n)$ time, as proved in the lecture. Dealing with the `freq` array with the for loop take $O(n)$ time. Dealing with the `validLen` array with the while loop takes $O(n)$ time since it looks at every element in the string at most once. The last for loop that calculates the final count with the for loop takes $O(n)$ times as well. Overall, the time complexity is $O(n)$.

Space complexity:

We created three arrays: `pi`, `validLen`, `freq`, all of which are $O(n)$ size. Every other variables take $O(1)$ space, resulting in an overall space complexity of $O(n)$.

Problem 2

2-1

4, 3, 5, 7, 9, 11, 13, 15, 17, 19, 20, 10, 2, 12, 6, 14, 1, 16, 8, 18

2-2

```
1 PartialInversePair(array A, x, k)
2     n = A.length
3     cnt = 0
4     ptrfront = 1
5     ptrback = x + 1
6     while ptrfront <= x and ptrback <= n
7         if A[ptrfront] > k * A[ptrback]
8             cnt += x - ptrfront + 1
9             ptrback++
10        else
11            ptrfront++
12    return cnt
```

Explanation:

Since the two subarrays is sorted non decreasingly, if (i, j) is a partial inverse pair(PIP), $(i + 1, j), (i + 2, j) \dots (x, j)$ will also be PIPs, so when if $A[i] > k \times A[j]$ holds, we can move to the next element of subarray $A[x+1 : n]$. If $A[i] > k \times A[j]$ does not hold, $(i, j+1), (i, j+2) \dots (i, n)$ will not be PIPs, so we can move on to the next element of the subarray $A[1 : x]$.

Complexity:

We scan through each element in array A only once, resulting in a $O(n)$ time and $O(1)$ space since we only use constant number of variables.

2-3

```
1 PartialInversePair(array A, low, mid, high, t)
2     cnt = 0
3     ptrfront = low
4     ptrback = mid + 1
5     while ptrfront <= mid and ptrback <= high
6         if A[ptrfront] > t * A[ptrback]
7             cnt += mid - ptrfront + 1
8             ptrback++
9         else
10            ptrfront++
11    return cnt
12
13 merge(B, low, mid, high)
14     newSizeB = high - low + 1
15     k = 1
16     i = low
17     j = mid + 1
18     newB = createArray(size = newSizeB)
19     while i <= mid and j <= high
20         if B[i] <= B[j]
21             newB[k++] = B[i++]
22         else
23             newB[k++] = B[j++]
24     while i <= mid
```

```

25         newB[k++] = B[i++]
26     while j <= high
27         newB[k++] = B[j++]
28     for i from 1 to newSizeB
29         B[low + i - 1] = newB[i]
30
31 SuperInversePair(array B, low, high, t)
32     if low >= high
33         return 0
34     mid = (low + high) / 2
35     leftCount = SuperInversePair(B, low, mid, t)
36     rightCount = SuperInversePair(B, mid + 1, high, t)
37     crossCount = PartialInversePair(B, low, mid, high, t)
38     merge(B, low, mid, high)
39     return leftCount + rightCount + crossCount

```

Explanation:

To use the `PartialInversePair` function, we need two sorted halves. We adapt Merge Sort to achieve this in $O(m \log m)$ time. Because standard merging only compares strict values ($B[i] \leq B[j]$), we must decouple the pair-counting logic from the sorting process. We recursively call `SuperInversePair` to divide the array, then use `PartialInversePair` to count the valid pairs across the two halves before they are merged. Doing this step before merging ensures the pairs we find still meet the original positional condition of $i < j$. The total count is simply the sum of all pairs found at every recursive layer.

Time complexity:

At each recursive layers, both the `PartialInversePair` and the merge function require takes $O(m)$ time. All other operations take $O(1)$ time. The total number of layers is $\log m$, resulting in a total time complexity of $O(m \log m)$.

Space complexity:

The total space malloc-ed by the merge function throughout the process is $m + m/2 + m/4 + \dots = O(2m) = O(m)$, and the total $\log m$ stack calls take up $O(\log m)$ spaces, resulting in $O(m) + O(\log m)$ space, satifying the $O(m \log m)$ constraint.

2-4

```

1 compare(s1, s2)
2     len = min(|s1|, |s2|)
3     for i = 1 to len
4         if s1[i] < s2[i]
5             return -1
6         if s1[i] > s2[i]
7             return 1
8     if |s1| < |s2|
9         return -1
10    else if |s1| > |s2|
11        return 1
12    else return 0
13
14 mergeSort(A, low, high)
15     if low >= high
16         return
17     mid = (low + high) / 2
18     mergeSort(A, low, mid)
19     mergeSort(A, mid + 1, high)
20     newSize = high - low + 1

```

```

21     newA = createArray(size = newSize)
22     leftPtr = low, rightPtr = mid + 1, k = 1
23     while leftPtr <= mid and rightPtr <= high
24         if compare(A[leftPtr], A[rightPtr]) <= 0
25             newA[k++] = A[leftPtr++]
26         else
27             newA[k++] = A[rightPtr++]
28     while leftPtr <= mid
29         newA[k++] = A[leftPtr++]
30     while rightPtr <= high
31         newA[k++] = A[rightPtr++]
32     for i = 1 to newSize
33         A[low + i - 1] = newA[i]
34     return A

```

Explanation:

Assume the set A contains pointers to each string. We simply apply merge sort on A , and do the comparison character by character.

Time complexity:

The merge sort recursion tree has $O(\log n)$ layers. As for the comparison, a single comparison takes $O(\min(|s_1|, |s_2|))$, and a string will only be added to the newA once per layer. Therefore, on each layer, the time spent on scanning the string will be $O(S_1 + S_2 + \dots + S_n) = O(m)$. Therefore, the overall time complexity will be $O(m \log n)$.

Space complexity:

Since there are m characters in total, storing the characters takes $O(m)$ space. The original pointer array take $O(n)$ space. During each merging process we will have to create an extra $O(n)$ space, but it will be freed after each iteration, so the space it take is $O(n)$. The stack call takes $O(\log n)$ space. Hence, the overall space complexity is $O(m + n + n + \log n) = O(m + n)$.

2-5

```

1 minHeapify(array T, i, heapSize)
2     left = 2 * i + 1
3     right = 2 * i + 2
4     smallest = i
5
6     if left < heapSize and T[left].score < T[smallest].score
7         smallest = left
8     if right < heapSize and T[right].score < T[smallest].score
9         smallest = right
10    if smallest != i
11        swap(T[i], T[smallest])
12        minHeapify(T, smallest, heapSize)
13
14
15 PMaximizer(array T, k)
16     n = |T|
17     mergeSort(T, by = speed, order = descending)
18     scoreSum = 0
19     heap = createArray(size = k)
20     for i from 1 to k
21         heap[i] = T[i]
22         scoreSum += T[i].score
23     P = scoreSum * T[k].speed
24     minHeapify(heap, 1, k)

```

```

25     for i from k + 1 to n
26         if T[i].score > peek(heap)
27             scoreSum = scoreSum + T[i].score - pop(heap)
28             insert(heap, T[i], 1, k)
29             newP = scoreSum * T[i].speed
30             if newP > P
31                 P = newP
32     return P

```

Explanation:

Since the multiplier is the minimum speed, we first sort T by speed in descending order. Then we create a min-heap to dynamically include the highest scores we see so far and take the minimum speed into consideration to derive the best score and combination so far.

Time complexity:

The merge sort take $O(n \log n)$ time. The first for loop takes $O(k)$ time to store the first k data. The second for loop runs for $n - k$ iterations, while the insert process takes $O(\log k)$ time, so this for loop costs $O((n - k) \times \log k)$ time. Overall, the time complexity is $O(n \log n + k + (n - k) \times \log k) = O(n \log n)$.

Space complexity:

The array "heap" takes $O(k) = O(n)$ space. Other than that, other variables take $O(1)$ space. The overall space complexity meets the $O(n)$ requirement.